

# Перебирающие методы массивов

---

## Функция как значение

Функцию можно сохранить в переменную или передать как аргумент — так же, как число, строку или массив. Если вызвать функцию с круглыми скобками `greet()` — получаем результат её выполнения. Без скобок `greet` — получаем ссылку на саму функцию.

```
function greet(name) {  
  return `Welcome ${name}!`;  
}  
  
console.log(greet("Mango")); // "Welcome Mango!" — результат вызова  
console.log(greet); // f greet() {...} — сама функция
```

## Колбек-функции (callback functions, колбеки)

**Колбек-функция (callback)** — функция, которая передаётся в другую функцию как аргумент, а та вызывает её внутри себя.

**Функция высшего порядка (higher order function)** — функция, которая принимает другую функцию как параметр или возвращает функцию в качестве результата.

```
function greet(name) {  
  console.log(`Welcome ${name}!`);  
}  
  
function notify(name) {  
  console.log(`Dear ${name}, your room will be ready in 30 minutes`);  
}  
  
// registerGuest — функция высшего порядка, callback — параметр для функции-колбека  
function registerGuest(name, callback) {  
  console.log(`Registering ${name}!`);  
  callback(name);  
}  
  
registerGuest("Mango", greet);  
// "Registering Mango!"  
// "Welcome Mango!"  
  
registerGuest("Mango", notify);  
// "Registering Mango!"  
// "Dear Mango, your room will be ready in 30 minutes"
```

Имя параметра для колбека может быть любым — важно, что значением будет функция.

## Инлайн-колбеки (inline callbacks)

Если колбек маленький и нужен только для передачи в конкретный вызов, его можно объявить прямо на месте — без отдельного объявления функции. Такая функция доступна только как значение параметра.

```
function registerGuest(name, callback) {
  console.log(`Registering ${name}!`);
  callback(name);
}

registerGuest("Mango", function greet(name) {
  console.log(`Welcome ${name}!`);
});
```

## Метод `forEach()`

```
array.forEach(function callback(element, index, array) {
  // тело колбек-функции
});
```

Перебирает массив поэлементно и вызывает колбек для каждого элемента. Всегда возвращает `undefined`, даже если внутри колбека написать `return`.

Колбек получает три параметра (порядок фиксирован, но имена — любые):

| Параметр             | Значение                  |
|----------------------|---------------------------|
| <code>element</code> | текущий элемент массива   |
| <code>index</code>   | его индекс                |
| <code>array</code>   | ссылка на исходный массив |

Если нужен только `element` (самый частый случай) — остальные параметры можно не объявлять.

```
const numbers = [5, 10, 15, 20, 25];

// Классический for
for (let i = 0; i < numbers.length; i += 1) {
  console.log(`Index ${i}, value ${numbers[i]}`);
}

// forEach
numbers.forEach(function (number, index) {
  console.log(`Index ${index}, value ${number}`);
});
```

**Важно:** `forEach` нельзя прервать досрочно (в отличие от `for` / `for...of`) — он всегда проходит массив до конца. Если нужен ранний выход из цикла — используйте `for` или `for...of`.

## Стрелочные функции

### Синтаксис

**Стрелочные функции (arrow functions, «стрелки»)** — сокращённый синтаксис объявления функций, уменьшающий объём кода. Особенно удобны для маленьких функций и колбеков. Стрелка всегда создаётся как функциональное выражение и обязательно присваивается переменной.

```
// Обычное объявление функции
function classicAdd(a, b, c) {
  return a + b + c;
}

// То же самое стрелочной функцией
const arrowAdd = (a, b, c) => {
  return a + b + c;
};
```

Ключевое слово `function` не используется — сразу указываются параметры, затем `=>` и тело функции в фигурных скобках.

- Несколько параметров — перечисляются через запятую в круглых скобках: `(a, b, c) =>`
- Один параметр — скобки можно опустить: `a =>`
- Без параметров — скобки обязательны и пустые: `() =>`

```
// Один параметр — без скобок
const add = a => {
  return a + 5;
};

// Без параметров — пустые скобки обязательны
const greet = () => {
  console.log("Hello!");
};
```

## Неявное возвращение (implicit return)

Тело стрелочной функции можно записать двумя способами.

### С фигурными скобками — явное возвращение (explicit return)

Если нужно выполнить в теле функции что-то ещё, кроме возврата значения, используются фигурные скобки, а значение возвращается через `return` явно.

```
const add = (a, b, c) => {
  console.log(a, b, c);
  return a + b + c;
};
```

### Без фигурных скобок — неявное возвращение (implicit return)

Если скобок нет, автоматически возвращается результат выражения, стоящего после `=>`. Слово `return` не пишется. Такой синтаксис уместен, только когда в теле функции не нужно ничего, кроме возврата значения.

```
const add = (a, b, c) => a + b + c;
```

```
// До
function classicAdd(a, b, c) {
  return a + b + c;
}

// После
const arrowAdd = (a, b, c) => a + b + c;
```

## Псевдомассив arguments

У стрелочных функций нет собственной локальной переменной `arguments` (псевдомассив всех аргументов, доступный в обычных функциях). Чтобы собрать все переданные аргументы в массив, используется оператор **rest (остаточные параметры)** — `...args`.

```
const add = (...args) => {
  console.log(args);
};

add(1, 2, 3); // [1, 2, 3]
```

## Стрелочные функции как колбеки

Анонимные стрелочные функции хорошо подходят для колбеков перебирающих методов массива благодаря короткому синтаксису — особенно если тело функции небольшое.

```
const numbers = [5, 10, 15, 20, 25];

// Обычная анонимная функция
numbers.forEach(function (number, index) {
  console.log(`Index ${index}, value ${number}`);
});

// Стрелочная анонимная функция
numbers.forEach((number, index) => {
  console.log(`Index ${index}, value ${number}`);
});
```

Стрелочный колбек можно объявить отдельно и передать по ссылке — это стоит делать, если функция используется в нескольких местах программы или её тело громоздкое.

```
const numbers = [5, 10, 15, 20, 25];

const logMessage = (number, index) => {
  console.log(`Index ${index}, value ${number}`);
};

numbers.forEach(logMessage);
```

## Методы map и flatMap

### Чистые функции

**Функция с побочными эффектами (side effects)** — функция, которая во время выполнения может: изменять или использовать глобальные переменные, изменять значения аргументов ссылочного типа (объекты, массивы), выполнять операции ввода-вывода и т.д.

```
const dirtyMultiply = (array, value) => {
  for (let i = 0; i < array.length; i += 1) {
    array[i] = array[i] * value;
  }
};

const numbers = [1, 2, 3, 4, 5];
dirtyMultiply(numbers, 2);
// Произошла мутация исходных данных — массива numbers
console.log(numbers); // [2, 4, 6, 8, 10]
```

**Чистая функция (pure function)** — функция, результат которой зависит только от переданных аргументов. При одинаковых аргументах она всегда возвращает один и тот же результат и не имеет побочных эффектов — не изменяет значения аргументов.

```
const pureMultiply = (array, value) => {
  const newArray = [];

  array.forEach(element => {
    newArray.push(element * value);
  });

  return newArray;
};

const numbers = [1, 2, 3, 4, 5];
const doubledNumbers = pureMultiply(numbers, 2);

// Мутации исходных данных не произошло
console.log(numbers); // [1, 2, 3, 4, 5]
// Функция вернула новый массив с изменёнными данными
console.log(doubledNumbers); // [2, 4, 6, 8, 10]
```

## Перебирающие методы: общий синтаксис

Большинство перебирающих методов массивов в JS — чистые функции. Они создают новый массив, заполняют его результатом применения колбека к каждому элементу и возвращают этот новый массив, не трогая исходный.

```
array.method(callback(currentValue, index, array))
```

Колбек-функция чаще всего получает три параметра (все, кроме первого, необязательны, имена — любые):

| Параметр                        | Значение                           |
|---------------------------------|------------------------------------|
| 1-й — <code>currentValue</code> | значение текущего элемента массива |
| 2-й — <code>index</code>        | индекс текущего элемента           |
| 3-й — <code>array</code>        | ссылка на исходный массив          |

```
// Если нужен только элемент
array.method(item => {
  // логика на каждой итерации
});
```

## Метод map()

`map(callback)` используется для трансформации массива: вызывает колбек для каждого элемента исходного массива, а результат записывает в новый массив — он и есть результат работы метода.

- Поэлементно перебирает исходный массив
- Не изменяет исходный массив
- Возвращает новый массив той же длины

```
const planets = ["Earth", "Mars", "Venus", "Jupiter"];

const planetsInUpperCase = planets.map(planet => planet.toUpperCase());
console.log(planetsInUpperCase); // ["EARTH", "MARS", "VENUS", "JUPITER"]

// Оригинальный массив не изменился
console.log(planets); // ["Earth", "Mars", "Venus", "Jupiter"]
```

## Массив объектов

Частая задача — достать из массива объектов массив значений одного свойства.

```
const students = [
  { name: "Mango", score: 83 },
  { name: "Poly", score: 59 },
  { name: "Ajax", score: 37 },
];

const names = students.map(student => student.name);
console.log(names); // ["Mango", "Poly", "Ajax"]
```

## Метод flatMap()

`flatMap(callback)` аналогичен `map()`, но применяется, когда результат — многомерный массив, который нужно «сгладить» (уменьшить вложенность).

```
const students = [
  { name: "Mango", courses: ["mathematics", "physics"] },
  { name: "Poly", courses: ["science", "mathematics"] },
  { name: "Kiwi", courses: ["physics", "biology"] },
];

const mappedCourses = students.map(student => student.courses);
console.log(mappedCourses);
// [["mathematics", "physics"], ["science", "mathematics"], ["physics", "biology"]]

const flattenedCourses = students.flatMap(student => student.courses);
console.log(flattenedCourses);
// ["mathematics", "physics", "science", "mathematics", "physics", "biology"]
```

`flatMap` вызывает колбек для каждого элемента и записывает результат в новый массив — как `map()`. Отличие: итоговый массив «сглаживается» на глубину в один уровень вложенности.

**map vs flatMap:** `map()` — колбек возвращает по одному значению на элемент, массивы вложенности не меняются. `flatMap()` — колбек может вернуть массив, и все такие массивы объединяются в один плоский результат.

## Методы filter и find

### Метод filter()

`filter(callback)` используется для фильтрации массива — отбора элементов коллекции по определённому критерию.

```
array.filter((element, index, array) => {  
  // тело колбек-функции  
});
```

- Не изменяет исходный массив, поэлементно его перебирает
- Возвращает новый массив
- Если колбек вернул `true` — элемент попадает в новый массив, если `false` — не попадает
- Если ни один элемент не подошёл — возвращает пустой массив

```
const values = [51, -3, 27, 21, -68, 42, -37];  
  
const positiveValues = values.filter(value => value >= 0);  
console.log(positiveValues); // [51, 27, 21, 42]  
  
const bigValues = values.filter(value => value > 1000);  
console.log(bigValues); // [] — ни один элемент не подошёл  
  
console.log(values); // [51, -3, 27, 21, -68, 42, -37] — оригинал не изменился
```

### filter() на массиве объектов

При работе с массивом объектов фильтрация обычно идёт по значению конкретного свойства.

```
const LOW_SCORE = 50;  
const HIGH_SCORE = 80;  
const students = [  
  { name: "Mango", score: 83 },  
  { name: "Poly", score: 59 },  
  { name: "Ajax", score: 37 },  
];  
  
const best = students.filter(student => student.score >= HIGH_SCORE);  
// объекты с Mango  
  
// В колбеке удобно деструктурировать свойства объекта  
const average = students.filter(  
  ({ score }) => score >= LOW_SCORE && score < HIGH_SCORE  
);  
// объекты с Poly
```

## Метод find()

Если `filter()` ищет **все** элементы, удовлетворяющие условию, то `find(callback)` находит и возвращает только **первый** подходящий элемент — после чего перебор массива останавливается.

```
array.find((element, index, array) => {  
  // тело колбек-функции  
});
```

- Не изменяет исходный массив, поэлементно его перебирает
- Возвращает первый элемент, для которого колбек вернул `true`
- Если подходящих элементов нет — возвращает `undefined`

Используется для поиска одного конкретного элемента: пользователя по email, товара по id и т.п.

```
const colorPickerOptions = [  
  { label: "red", color: "#F44336" },  
  { label: "green", color: "#4CAF50" },  
  { label: "blue", color: "#2196F3" },  
];  
  
colorPickerOptions.find((option) => option.label === "blue");  
// { label: "blue", color: "#2196F3" }  
  
colorPickerOptions.find((option) => option.label === "white");  
// undefined — совпадений нет
```

## Методы every, some и reduce

---

### Метод every()

`every(callback)` проверяет, удовлетворяют ли **все** элементы массива условию колбека.

```
array.every((element, index, array) => {  
  // тело колбек-функции  
});
```

- Не изменяет исходный массив, поэлементно его перебирает
- Возвращает `true`, если все элементы подошли под условие
- Возвращает `false`, если хотя бы один элемент не подошёл
- Перебор останавливается, как только колбек вернул `false`

```
// Все элементы >= 0? — да  
[1, 2, 3, 4, 5].every((value) => value >= 0); // true  
  
// Все элементы >= 0? — нет  
[1, 2, 3, -10, 4].every((value) => value >= 0); // false
```



```
const products = [
  { name: "apple", quantity: 2 },
  { name: "orange", quantity: 5 },
  { name: "plum", quantity: 0 },
];

const hasEveryProduct = products.every(product => product.quantity > 0);
console.log(hasEveryProduct); // false — plum закончился
```

## Метод some()

`some(callback)` проверяет, удовлетворяет ли условию колбека **хотя бы один** элемент массива.

```
array.some((element, index, array) => {
  // тело колбек-функции
});
```

- Не изменяет исходный массив, поэлементно его перебирает
- Возвращает `true`, если хотя бы один элемент подошёл под условие
- Возвращает `false`, если ни один элемент не подошёл
- Перебор останавливается, как только колбек вернул `true`

```
// Есть хотя бы один элемент >= 0? — да
[-7, -20, 3, -10].some(value => value >= 0); // true

// Есть хотя бы один элемент < 0? — нет
[1, 2, 3, 4, 5].some(value => value < 0); // false
```

**every vs some:** `every()` — верно ли условие для **всех** элементов. `some()` — верно ли условие хотя бы для **одного** элемента.

## Метод reduce()

`reduce(callback, initialValue)` последовательно обрабатывает каждый элемент массива, сохраняя промежуточный результат (аккумулятор). Самый гибкий из перебирающих методов — им можно заменить функционал любого другого метода массива, включая их комбинации.

```
array.reduce((previousValue, element, index, array) => {
  // тело колбек-функции
}, initialValue);
```

- Не изменяет исходный массив, поэлементно его перебирает
- Может вернуть что угодно: объект, массив, строку, число

`reduce()` принимает 2 параметра: колбек-функцию (обязательный) и `initialValue` — начальное значение аккумулятора (необязательный).

Колбек получает 4 параметра (порядок фиксирован, необязательные можно опускать):

| Параметр                         | Значение                                                                                                                        |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| 1-й — <code>previousValue</code> | аккумулятор — промежуточный результат. То, что колбек вернул на текущей итерации, станет значением этого параметра на следующей |
| 2-й — <code>element</code>       | текущий элемент массива                                                                                                         |
| 3-й — <code>index</code>         | индекс текущей итерации                                                                                                         |
| 4-й — <code>array</code>         | ссылка на исходный массив                                                                                                       |

```
const total = [2, 7, 3].reduce((previousValue, number) => {
  return previousValue + number;
}, 0);

console.log(total); // 12
```

Как это работает пошагово:

- Начальное значение аккумулятора — `0`
- 1-я итерация: `0 + 2` → возвращает `2`
- 2-я итерация: `2 + 7` → возвращает `9`
- 3-я итерация: `9 + 3` → возвращает `12`

`reduce()` применяют, когда нужно «взять много и получить одно». В повседневных задачах это чаще всего работа с числами (суммы, средние значения и т.п.).

## reduce() и массив объектов

При работе с массивом объектов редуцирование обычно идёт по значению конкретного свойства. Например, нужно посчитать средний балл студентов.

```
const students = [
  { name: "Mango", score: 83 },
  { name: "Poly", score: 59 },
  { name: "Ajax", score: 37 },
];

// Имя аккумулятора произвольное — это просто параметр функции
const totalScore = students.reduce((total, student) => {
  return total + student.score;
}, 0);

const averageScore = totalScore / students.length;
```

`reduce()` здесь суммирует значения `score` всех объектов, начиная с `0`. Результат — `totalScore`. Средний балл `averageScore` получается делением суммы на количество студентов.

# Метод toSorted

## Метод toSorted()

`toSorted()` сортирует элементы массива.

```
array.toSorted();
```

- Не изменяет исходный массив
- Возвращает новый отсортированный массив
- По умолчанию сортирует по возрастанию

## Массив чисел

```
const scores = [61, 19, 74, 35, 92, 56];
const ascendingScores = scores.toSorted();

console.log(scores); // [61, 19, 74, 35, 92, 56] — не изменился
console.log(ascendingScores); // [19, 35, 56, 61, 74, 92]
```

**Важно:** по умолчанию `toSorted()` перед сравнением приводит элементы к строкам — числа сортируются как строки (по таблице Unicode), а не как числа.

```
const scores = [27, 2, 41, 4, 7, 3, 75];

console.log(scores.toSorted()); // [2, 27, 3, 4, 41, 7, 75] — сортировка как строк!
```

Строки сравниваются посимвольно слева направо: сначала «2» и «27» — первый символ одинаковый, но «2» короче, значит меньше и идёт первым. Затем «27» и «3» — первый символ «3» больше «2», поэтому «3» идёт после «27». Как задать свой (числовой) порядок сортировки — ниже.

## Массив строк

Массив строк сортируется по алфавиту. При этом код заглавных букв меньше, чем строчных.

```
const students = ["Jacob", "Artemis", "Adrian", "Kai"];
console.log(students.toSorted()); // ["Adrian", "Artemis", "Jacob", "Kai"]

const letters = ["b", "B", "a", "A"];
console.log(letters.toSorted()); // ["A", "B", "a", "b"]
```

## Свой порядок сортировки чисел

Чтобы задать свой порядок, в `toSorted(compareFunction)` передаётся **функция сравнения (compare function)** с двумя параметрами — метод вызывает её для пар элементов.

```
array.toSorted((a, b) => {
  // тело функции сравнения
});
```

- Если `compareFunction(a, b)` вернула отрицательное число (`a` меньше `b`) — `a` ставится перед `b`
- Если вернула положительное число (`b` больше `a`) — `b` ставится перед `a`
- Если вернула `0` — порядок `a` и `b` друг относительно друга не меняется

```
const scores = [61, 19, 74, 35, 92, 56];

// По возрастанию
const ascendingScores = scores.sort((a, b) => a - b);
// [19, 35, 56, 61, 74, 92]

// По убыванию
const descendingScores = scores.sort((a, b) => b - a);
// [92, 74, 61, 56, 35, 19]
```

При передаче функции сравнения числа больше не приводятся к строкам — сортировка чисел работает как ожидается.

## Свой порядок сортировки строк

Для сортировки строк по алфавиту (по возрастанию или убыванию) используется строковый метод `localeCompare()`.

```
firstString.localeCompare(secondString)
```

- Возвращает отрицательное число, если `firstString` должен идти перед `secondString`
- Возвращает положительное число, если `firstString` должен идти после `secondString`
- Возвращает `0`, если строки равны

```
"a".localeCompare("b"); // -1
"b".localeCompare("a"); // 1
"a".localeCompare("a"); // 0
```

`localeCompare()` удобно использовать в `sorted()`, потому что она ожидает от колбека такие же значения (отрицательное / положительное / ноль).

```
const students = ["Jacob", "Artemis", "Adrian", "Kai"];

const inAlphabetOrder = students.sort((a, b) => a.localeCompare(b));
// ["Adrian", "Artemis", "Jacob", "Kai"]

const inReversedOrder = students.sort((a, b) => b.localeCompare(a));
// ["Kai", "Jacob", "Artemis", "Adrian"]
```

## Сортировка объектов

При сортировке массива объектов функция сравнения строится на значении конкретного свойства.

```

const students = [
  { name: "Mango", score: 83 },
  { name: "Poly", score: 59 },
  { name: "Ajax", score: 37 },
  { name: "Kiwi", score: 94 },
];

// По возрастанию баллов
const inAscendingScoreOrder = students.sorted(
  (a, b) => a.score - b.score
);

// По убыванию баллов
const inDescendingScoreOrder = students.sorted(
  (a, b) => b.score - a.score
);

// По алфавиту (имя)
const inAlphabeticalOrder = students.sorted((a, b) =>
  a.name.localeCompare(b.name)
);

```

## Цепочки методов (method chaining)

Массив методов можно объединять в цепочку: каждый следующий метод выполняется над результатом предыдущего. Это избавляет от лишних промежуточных переменных.

```

const students = [
  { name: "Mango", score: 83, courses: ["mathematics", "physics"] },
  { name: "Poly", score: 59, courses: ["science", "mathematics"] },
  { name: "Ajax", score: 37, courses: ["physics", "biology"] },
];

// Без цепочки — лишняя промежуточная переменная
const sortedByAscendingScore = students.sorted((a, b) => a.score - b.score);
const names = sortedByAscendingScore.map(student => student.name);

// С цепочкой — без лишней переменной
const names2 = students
  .sorted((a, b) => a.score - b.score)
  .map(student => student.name);

console.log(names2); // ["Ajax", "Poly", "Mango"]

```

Более длинный пример: получить отсортированный по алфавиту список уникальных предметов, которые изучают студенты.

```

const uniqueSortedCourses = students
  .flatMap(student => student.courses)
  .filter((course, index, array) => array.indexOf(course) === index)
  .sorted((a, b) => a.localeCompare(b));

console.log(uniqueSortedCourses); // ["biology", "mathematics", "physics", "science"]

```

1. `flatMap()` — собирает все курсы всех студентов в один плоский массив
2. `filter()` — оставляет только уникальные значения (элемент проходит, только если его первое вхождение по `indexOf` совпадает с текущим индексом)
3. `sorted()` — сортирует результат по алфавиту

**Важно:** цепочка может быть любой длины, но обычно не длиннее 2–3 методов — каждый следующий метод в цепочке заново перебирает массив, что при больших объёмах данных влияет на производительность.